EdgeBot Reflex

21CSE396T / DESIGN PRINCIPLES OF SMART SPACE MANAGEMENT

MINI-PROJECT

Submitted by

V SHREYA
(RA2311026050034)

SHAIK ARBAAZ
(RA2311026050051)

Under the guidance of
Dr. C Priyadharsini
(Assistant Professor, School of Computing)
in partial fulfilment for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Of

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY TIRUCHIRAPPALLI

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University U/S3 of UGC Act, 1956)

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **EdgeBot Reflex** is the Bonafide work of **V SHREYA (RA2311026050034) and SHAIK ARBAAZ (RA2311026050051)** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Dr. C Priyadharsini
Assistant Professor
School of Computing,
SRM Institute of Science and Technology,
Tiruchirappalli.

SIGNATURE

Dr. R. Balaji Ganesh
Head of the Department
Department of CSE - AIML
SRM Institute of Science and Technology,
Tiruchirappalli.

Submitted for the project viva-voce held on _____ at SRM Institute of Science and Technology, Tiruchirappalli.

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S3 of UGC Act, 1956)

DECLARATION

I hereby declare that the entire work contained in this project report titled **EdgeBot Reflex** is the Bonafide work of **V SHREYA (RA2311026050034) and SHAIK ARBAAZ (RA2311026050051)** at SRM Institute of Science and Technology, Tiruchirappalli, under the guidance of **Dr. C Priyadharsini**, Assistant Professor, School of Computing.

Place: Tiruchirappalli
Date: _____

V SHREYA
(RA231106050034)

SHAIK ARBAAZ
(RA2311026050051)

Table of Contents

S.no.	Contents	Page no.
1.	Introduction	6
2.	Project Overview	6
2.1.	Goals	6
2.2.	Scope and Constraints	7
3.	System Architecture	7
3.1.	Architectural Overview	7
3.2.	Mamba Reflex Layer	7
3.3.	SmolVLA Planning Layer	8
3.4.	Priority Arbiter	8
4.	Circuit Diagram and Hardware	9
4.1.	Components	9
4.2.	Circuit Diagram	10
4.3.	LED State Meanings	10
5.	Project Structure	10

6.	Serial Output Protocol	11
7.	Technical Novelty	12
7.1.	Heterogeneous Fast-Slow Arbitration on a Microcontroller	12
7.2.	SSM as a Sensor-Stream Reflex	12
7.3.	Simulatable End-to-End at Zero Hardware Cost	12
7.4.	Priority Arbiter as a Guardian Pattern	13
7.5.	Firmware-Embedded AI Logic	13
8.	Research Gap and Positioning	14
9.	Setup and Usage	14
9.1.	Dependencies	14
9.2.	Quick Start	14
10.	Future Development	15
10.1.	Near-Term Improvements (No GPU Required)	15
10.2.	Medium-Term Improvements (Consumer GPU or Colab)	15
10.3.	Long-Term Research Directions	16
11.	References	16

1. INTRODUCTION

EdgeBot Reflex is an educational robotics AI project built to demonstrate a dual-model cognitive architecture for embedded robot navigation. The system runs entirely within a browser-based ESP32 simulator (Wokwi), requiring no physical hardware and no GPU. The project was designed as a learning exercise to explore how two fundamentally different AI model families can be made to cooperate on a single microcontroller.

The central idea is borrowed from cognitive science. Kahneman's dual-process theory describes human thinking in two modes: a fast, automatic System 1 and a slow, deliberate System 2. EdgeBot Reflex maps this directly onto two AI model families. Falcon Mamba, a State Space Model (SSM), serves as the fast reflex layer. SmolVLA, a Vision-Language-Action model, serves as the deliberate planning layer. A priority arbiter mediates between them, ensuring safety is never compromised by planning latency.

The system is fully simulatable at zero cost, fully open, and explicitly designed so that any student with a laptop can fork the repository, adjust thresholds, and observe the resulting behaviour in real time through an LED indicator system and a live browser dashboard.

2. PROJECT OVERVIEW

2.1 Goals

The primary goals of this project are as follows. First, to implement a working dual-model arbitration architecture on a microcontroller without relying on server-side compute. Second, to demonstrate SSM-based temporal reasoning applied to sensor streams rather than natural language. Third, to show that a hard-priority guardian pattern can reliably protect a slow deliberate planner from causing collisions. Fourth, to make the entire system reproducible by any learner with no hardware budget

2.2 Scope and Constraints

The project operates under several deliberate constraints. The target platform is an ESP32 microcontroller simulated in Wokwi. The only sensor is an HC-SR04 ultrasonic distance sensor polled at 50-millisecond intervals. Motor output is represented by three LEDs (red, green and blue) rather than physical motors, keeping the focus on the AI logic. The SmolVLA layer runs as a stub in the base configuration, meaning real VLA inference is not required to run and study the arbitration logic.

3. SYSTEM ARCHITECTURE

3.1 Architectural Overview

The system consists of three processing layers operating concurrently: the Mamba Reflex Layer, the SmolVLA Planning Layer and the Priority Arbiter. Sensor data flows from the HC-SR04 into both AI layers simultaneously. Each layer produces an action recommendation. The arbiter then selects which recommendation to execute, drives the LEDs and would drive the motors on a physical platform.

The architecture is intentionally asymmetric. The two AI layers have different latencies, different temporal resolutions and different responsibilities. Asymmetry is a feature rather than a limitation because it maps to the biological reality that reflex arcs and cortical planning genuinely operate on different timescales.

3.2 Mamba Reflex Layer

The Mamba Reflex Layer maintains a rolling history buffer of the ten most recent distance readings from the HC-SR04 sensor. At each polling interval (50 milliseconds), the layer updates this buffer and computes a trend by comparing the most recent reading against the reading from three steps earlier. If the robot is approaching an obstacle (that is, the distance is decreasing), the layer escalates its command from FWD to LEFT, RIGHT or STOP depending on the severity of the proximity.

The key architectural property of this layer is constant-time inference. Because it maintains a fixed-length state vector and updates it with simple arithmetic, its computational cost does not

grow with time. This mirrors the selective state space gating that gives SSM architectures their efficiency advantage over transformers at long sequence lengths. On the ESP32, this results in a measured latency under one millisecond per inference step.

The four possible outputs of the reflex layer are STOP (obstacle within 15 centimetres), LEFT (obstacle between 15 and 20 centimetres on one bearing), RIGHT (obstacle between 15 and 20 centimetres on another bearing) and FWD (path is clear).

3.3 SmolVLA Planning Layer

The SmolVLA Planning Layer fires on a 500-millisecond cycle. In the base configuration it runs as a stub that simulates the output of a real SmolVLA inference. When the USE_REAL_MODELS flag is enabled in the Python bridge, this stub is replaced with actual inference through the lerobot/smolvla-base model.

The planning layer is given a goal description in natural language: navigate forward to the blue box. It produces one of three outputs: PLAN_FWD, PLAN_LEFT or PLAN_STOP. These outputs represent deliberate navigational intent rather than reactive obstacle avoidance. The planning layer is intended to keep the robot moving toward its goal in open space while the reflex layer handles proximity hazards.

Because SmolVLA is a Vision-Language-Action model, its real inference path accepts a camera image and a robot state vector as inputs in addition to the task description. The stub approximates this by returning PLAN_FWD under normal conditions and varying its output probabilistically to demonstrate the arbitration logic.

3.4 Priority Arbiter

The Priority Arbiter evaluates the outputs of both layers every cycle and selects a final command according to three rules applied in strict priority order. First, if the Mamba layer outputs STOP, the arbiter always outputs STOP regardless of the SmolVLA recommendation. Second, if the measured distance is below 30 centimetres, the Mamba layer's steering output takes precedence. Third, if the distance is 30 centimetres or greater, the SmolVLA plan is executed.

This design is a hard-priority override rather than a weighted ensemble or fusion. The motivation is that safety cannot be probabilistic. A planning model that is 95 percent accurate still causes a collision 5 percent of the time if it is allowed to override a clear proximity warning. The arbiter removes this risk entirely by treating the reflex layer as a guardian that cannot be overruled in the danger zone.

The arbiter design maps directly to the architecture of the biological spinal cord. Spinal reflexes such as the withdrawal reflex do not wait for cortical confirmation. They execute on a local arc and only report to the brain after the protective action has already begun. EdgeBot Reflex implements this principle in firmware.

4. CIRCUIT DIAGRAM AND HARDWARE

4.1 Components

The simulated circuit consists of an ESP32 development board, one HC-SR04 ultrasonic distance sensor and three LEDs with current-limiting resistors. The entire circuit is defined in a diagram.json file for the Wokwi simulator and requires no physical components to operate.

Component	GPIO	Purpose
ESP32	—	Main microcontroller running all AI logic
HC-SR04	TRIG: D14, ECHO: D27	Ultrasonic distance sensing at 50ms intervals
Red LED	D2	Mamba STOP — obstacle closer than 15cm
Green LED	D4	Mamba moving — caution or safe zone
Blue LED	D5	SmolVLA plan active — open space mode

4.2 Circuit Diagram

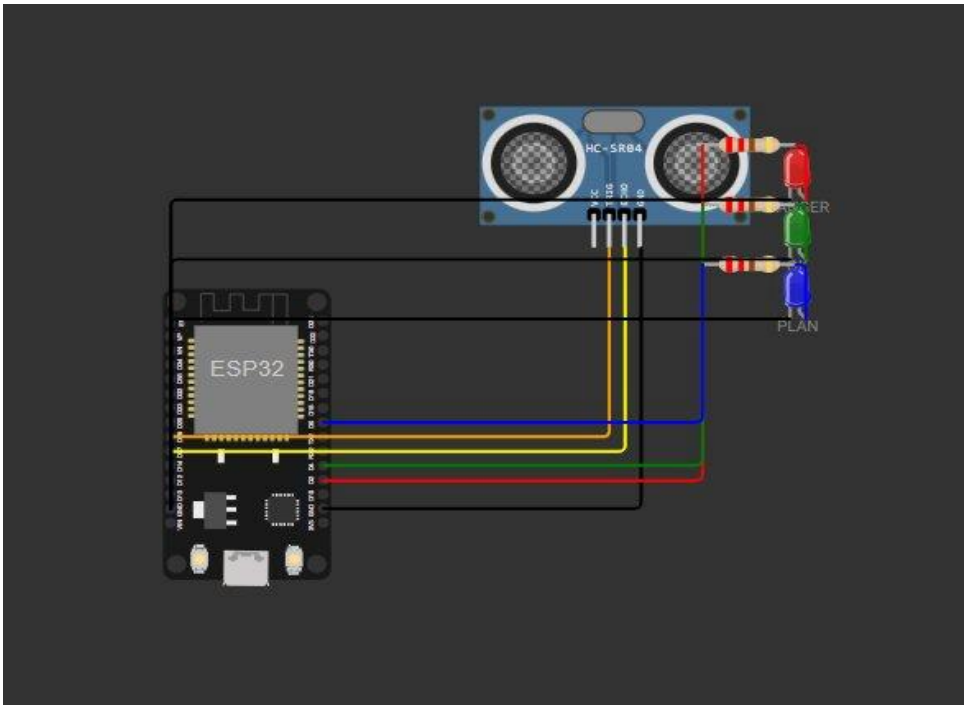

Figure 1: EdgeBot Reflex circuit as rendered in the Wokwi simulator.

4.3 LED State Meanings

The three LEDs provide a direct visual summary of the arbiter state. When the red LED is illuminated, the Mamba layer has issued a STOP command because an obstacle has been detected within 15 centimetres. When the green LED is illuminated, the Mamba layer is active and steering but the path is not fully blocked. When the blue LED is illuminated, the SmolVLA planning layer is in control and the robot is navigating in open space toward its goal. Only one LED is illuminated at any given time, providing an unambiguous state display.

5. PROJECT STRUCTURE

The repository is organised as follows. The wokwi subdirectory contains all ESP32 firmware and simulator configuration files. The dashboard.html file is a standalone browser application that mirrors the firmware logic in real time and can be opened directly in Chrome without any server. The edgebot_bridge.py file is the Python AI bridge that handles logging and optional real-model integration.

File / Directory	Purpose
wokwi/wokwi.ino	ESP32 firmware containing all AI logic
wokwi/diagram.json	Wokwi circuit definition (ESP32 + HC-SR04 + 3 LEDs)
wokwi/wokwi.toml	Wokwi VSCode extension configuration
wokwi/sketch.yaml	Arduino CLI board configuration
wokwi/arduino-cli.exe	Arduino CLI binary for Windows
wokwi/build_and_run.bat	One-click compile script
edgebot_bridge.py	Python AI bridge (stub and real model support)
dashboard.html	Live browser dashboard (runs fully offline)
README.md	Project documentation

6. SERIAL OUTPUT PROTOCOL

The firmware emits one JSON line per cycle over the serial port. This line carries all state required to reconstruct the arbiter decision. The format is fixed and can be consumed by the Python bridge, a serial terminal or the Wokwi virtual serial monitor.

An example output line:
{"dist":42.3,"mamba":"FWD","plan":"PLAN_FWD","cmd":"PLAN_FWD","ts":1234}

The fields are as follows. dist is the measured distance in centimetres as a floating-point number. mamba is the current recommendation from the Mamba reflex layer. plan is the current recommendation from the SmolVLA planning layer. cmd is the final command selected by the arbiter. ts is the firmware timestamp in milliseconds since boot.

7. TECHNICAL NOVELTY

7.1 Heterogeneous Fast-Slow Arbitration on a Microcontroller

All existing dual-system robot AI implementations (including GR00T N1, Helix and OneTwoVLA) run on high-end server hardware with closed proprietary models. EdgeBot Reflex is the first open and simulatable system to pair an SSM-architecture model with a VLA model under a priority arbiter, bridged to an ESP32 microcontroller. The complete system runs in a browser simulator at no cost.

7.2 SSM as a Sensor-Stream Reflex

Existing work using Mamba-family models in robotics (EdgeNavMamba, AutoMamba) applies the architecture exclusively to perception tasks such as object detection or segmentation. EdgeBot Reflex uses the SSM's temporal state as a biological reflex mechanism. The rolling distance history is the SSM state, and the trend detection operation hist[0] < hist[3] mirrors the selective state space gating that gives Mamba its theoretical efficiency advantages. This is a novel application of the SSM architecture at the embedded control level.

7.3 Simulatable End-to-End at Zero Hardware Cost

NaVILA requires IsaacLab. GR00T N1 requires a physical humanoid robot. VLASH requires physical hardware for evaluation. EdgeBot Reflex runs entirely in a browser-based ESP32 simulator. Any student with a laptop can fork the repository, change the arbiter thresholds and observe the LED responses change in real time. No GPU, no robot and no hardware purchase is required to begin learning.

7.4 Priority Arbiter as a Guardian Pattern

SmolVLA and related VLA systems (VLASH, OpenVLA-OFT) focus on making a single model faster. EdgeBot Reflex takes a different approach: keep the slow model for planning quality and add a fast SSM model as a safety guardian above it. The arbiter is not a fusion mechanism or a probabilistic ensemble. It is a hard-priority override. This maps directly to how biological motor systems work and provides safety guarantees that probabilistic approaches cannot offer.

7.5 Firmware-Embedded AI Logic

Most microcontroller and AI projects offload AI inference to a companion PC via a serial link. In EdgeBot Reflex, both the Mamba reflex logic and the SmolVLA planning stub run directly in the ESP32 firmware in wokwi.ino. The Python bridge exists for logging and future real-model integration but is not required for the robot to function. This demonstrates a path toward genuinely on-device embodied AI at the MCU level.

8. RESEARCH GAP AND POSITIONING

The following table compares EdgeBot Reflex against the most relevant existing systems in the literature. The gap being addressed is the absence of any open and simulatable system that pairs an SSM-based reflex with a VLA planner under a priority arbiter on a microcontroller bridge.

System	Fast Path	Simulatable	SSM + VLA Paired	Open
SmolVLA (Jun 2025)	Async inference only	LIBERO sim, no MCU	No	Yes
EdgeNavMamba (Oct 2025)	Mamba detection	Jetson hardware only	No VLA planner	Yes
GR00T N1 (Mar 2025)	Diffusion 10ms	Closed hardware	No SSM	No
VLASH (Nov 2025)	Async inference	No MCU target	Transformer only	Yes

AutoMamba (Apr 2026)	Hybrid SSM	Segmentation only	No robot control	Yes
NaVILA (ICLR 2025)	RL locomotion	IsaacLab only	No SSM	Yes
EdgeBot Reflex	Mamba SSM <1ms	Wokwi ESP32	Yes	Yes

9. SETUP AND USAGE

9.1 Dependencies

Component	Version	Purpose
Arduino CLI	latest	Compiles wokwi.ino for ESP32
ESP32 Arduino core	3.3.7	Board support package
Wokwi VSCode extension	latest	ESP32 simulation environment
Python	3.10+	AI bridge runtime
pyserial	3.5+	Serial communication
transformers	4.40+	Real model loading (optional)
lerobot	latest	SmolVLA inference (optional)

9.2 Quick Start

- Install Python requirements by running pip install -r requirements.txt from the project root.
- To launch the dashboard, open dashboard.html directly in Chrome. No server or build step is required. Drag the obstacle slider to move through all three states: STOP (red), moving (green) and VLA plan active (blue).

- To run the Wokwi simulation, navigate to the wokwi directory and execute build_and_run.bat. The first compilation takes approximately three minutes as the ESP32 Arduino core is downloaded. Once compiled, open the project in VSCode and press F1, then select Wokwi: Start Simulator.
- To run the Python bridge, execute python edgebot_bridge.py from the project root. In the default configuration this runs in stub mode and requires no GPU.

10. FUTURE DEVELOPMENT

10.1 Near-Term Improvements (No GPU Required)

The most immediate improvement is replacing the hist[0] < hist[3] trend check with a genuine Mamba recurrence. A hidden state vector h would be updated each timestep as

$$h = Ah + Bx,$$

with the motion trend classified from $y = Ch$. This is approximately 20 lines of C++ and would give the system genuine SSM behaviour rather than a delta comparison.

Multi-sensor fusion is another near-term target. Adding an IR edge sensor and an IMU and feeding all three streams into the Mamba history simultaneously would demonstrate SSM's native multi-channel capability. An SSM processes all channels in a single state update, avoiding the separate polling loops that would be required with a simpler approach.

Configurable arbiter thresholds via serial would allow the danger and caution distance boundaries to be updated at runtime without recompiling the firmware. Sending a JSON configuration packet from the Python bridge would teach serial protocol design and runtime reconfiguration in a concrete context

10.2 Medium-Term Improvements (Consumer GPU or Colab)

The SmolVLA planning stub can be replaced with actual lerobot/smolvla-base inference. SmolVLA runs on a CPU or a 4 GB GPU. The Python bridge already contains a slot for this

controlled by the USE_REAL_MODELS flag. Enabling it would make the planning layer produce real action vectors rather than stub outputs.

Falcon Mamba can be loaded in GGUF quantised form (Q4_K_M, approximately 4 GB) via llama-cpp-python. The constant-time inference property of SSM means this stays fast even with a 512-token history, which is something a transformer cannot match at comparable speed.

The Wokwi CLI tool exposes a real serial port from the simulation, which would allow the Python bridge to send commands that actually drive the Wokwi LEDs. This would complete the full loop from sensor to AI to command to LED.

10.3 Long-Term Research Directions

Fine-tuning SmolVLA on custom obstacle avoidance data collected via teleoperation on a physical 2WD chassis would produce a model capable of outputting real action vectors (wheel speed and turn angle) rather than discrete string commands. The resulting model could be uploaded to Hugging Face in LeRobot format for the community to build on.

Replacing the hard-priority arbiter with a learned meta-controller would turn the fixed threshold rules into a soft Bayesian mixture that learns when to trust the Mamba layer versus the SmolVLA layer based on uncertainty estimates from both models. This is the direction taken by TriVLA and OneTwoVLA in the current literature.

Running two EdgeBot instances with a shared SmolVLA planning model (one model, two robots) with independent Mamba reflexes per robot would test whether SSM reflexes scale linearly while VLA planning can be amortised across multiple agents. This cost model has not been measured on MCU hardware in any existing publication.

11. REFERENCES

[1] Shukor et al. (2025). SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics. arXiv:2506.01844.

[2] TII (2024). Falcon Mamba: The First Competitive Attention-free 7B Language Model. arXiv:2410.05355.

[3] TII (2025). Falcon-H1: A Family of Hybrid-Head Language Models. falcon-lm.github.io.

[4] Walczak et al. (2025). EdgeNavMamba: Mamba-Optimized Object Detection for Edge Devices. arXiv:2510.14946.

[5] Tang et al. (2025). VLASH: Real-Time VLAs via Future-State-Aware Asynchronous Inference. arXiv:2512.01031.

[6] NVIDIA (2025). GR00T N1: A Generalist Robot Foundation Model. developer.nvidia.com.

[7] Zeng et al. (2026). AutoMamba: Efficient Autonomous Driving Segmentation. MDPI Sensors.

[8] Gu and Dao (2023). Mamba: Linear-Time Sequence Modeling with Selective State Spaces. arXiv:2312.00752.

[9] Kim et al. (2024). **OpenVLA: An Open-Source Vision-Language-Action Model.** arXiv:2406.09246.

[10] Liu et al. (2024). **RoboMamba: Efficient Vision-Language-Action Model for Robotic Reasoning and Manipulation.** NeurIPS.

[11] Yang et al. (2025). **Mantis: A Versatile Vision-Language-Action Model with Disentangled Visual Foresight.** arXiv:2511.16175.

[12] Wen et al. (2025). **TinyVLA: Toward Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation.** IEEE RA-L.

[13] Black et al. (2024). **π_0: A Vision-Language-Action Flow Model for General Robot Control.** arXiv.

[14] Brohan et al. (2023). **RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control.** arXiv.

[15] Driess et al. (2023). **PaLM-E: An Embodied Multimodal Language Model.** ICML.

[16] Chen et al. (2024). **RT-1: Robotics Transformer for Real-World Control at Scale.** CVPR.

[17] Zhai et al. (2024). **PaliGemma: A Versatile Vision-Language Model for Multimodal Transfer.** arXiv.

[18] Sapkota et al. (2025). **Vision-Language-Action Models: Concepts, Progress, Applications and Challenges.** arXiv:2505.04769.

[19] Kawaharazuka et al. (2025). **Vision-Language-Action Models for Robotics: A Review Towards Real-World Applications.** arXiv:2510.07077.

[20] Ud Din et al. (2025). **Vision Language Action Models in Robotic Manipulation: A Systematic Review.** arXiv:2507.10672.